

umm API & MCP 연동 가이드 (API & MCP Guide)

umm은 시스템 통합 및 자동화를 위한 **REST API**와 AI 에이전트(Claude Desktop, Cursor, Custom Agent 등) 연동을 위한 **Model Context Protocol (MCP)** 엔드포인트를 제공합니다.

1. 인증 체계 (Authentication)

모든 API 및 MCP 호출은 개인 설정(`/settings`)에서 발급받은 **Bearer API Key** 또는 세션 쿠키를 사용합니다:

```
Authorization: Bearer umm_key_a1b2c3d4_XXXXXXXXXXXXXXXXXXXXXXXXXXXX
```

2. REST API 주요 엔드포인트 (`/api/v1`)

공간 (Spaces)

메서드	경로	설명	권한 스코프
GET	<code>/spaces</code>	사용자가 접근 가능한 공간 목록 조회	<code>spaces:read</code>
POST	<code>/spaces</code>	새 공간 생성 (<code>{"name": "새 프로젝트"}</code>)	<code>notes:write</code>
PUT	<code>/spaces/{id}</code>	공간 이름 변경 (<code>{"name": "수정된 이름"}</code>)	<code>notes:write</code>
DELETE	<code>/spaces/{id}</code>	공간 및 내부 메모 영구 삭제	<code>notes:write</code>
GET	<code>/spaces/{id}/members</code>	공간 참여 멤버 및 권한 목록 조회	<code>spaces:read</code>
POST	<code>/spaces/{id}/members</code>	공간에 사용자 초대	<code>notes:write</code>

📝 생각 포스트잇 (Notes & Edges)

메서드	경로	설명	권한 스코프
GET	/spaces/{id}/notes	공간 내 모든 포스트잇과 연결선 조회	notes:read
POST	/spaces/{id}/notes	새 포스트잇 생성 (title , content , color , x , y , width , height)	notes:write
PUT	/notes/{id}	포스트잇 내용/위치/크기 수정	notes:write
DELETE	/notes/{id}	포스트잇 삭제	notes:write
GET	/notes/{id}/history	포스트잇 과거 수정 이력 스냅샷 조회	notes:read
POST	/notes/{id}/restore/{version}	특정 버전으로 포스트잇 복원	notes:write
GET	/notes/{id}/related	의미 유사도 기반 연관 생각 목록 추천	notes:read
GET	/notes/{id}/backlinks	들어오고 나가는 실제 연결선과 상대 생각 조회	notes:read
GET	/search?q= ...	로컬 의미·키워드 하이브리드 검색과 필터 ·cursor	notes:read
GET	/notes/{id}/comments	댓글·멘션 목록	notes:read
POST	/notes/{id}/comments	댓글 작성과 멘션 알림	notes:write
PUT	/comments/{id}/resolve	댓글 해결·재개	notes:write
POST	/spaces/{id}/edges	두 포스트잇 간 연결선 생성 (source , target , relation)	notes:write

☀️ Today 리뷰

메서드	경로	설명	권한 스코프
GET	/today	검토 대상·고립 생각·댓글·온보딩 집계. Dream 항목은 별도 권한이 있을 때만 포함	notes:read (dreams:read for Dream)
POST	/notes/{id}/review	검토 완료, 다시 보기, 고정	notes:write
GET	/onboarding	실제 사용 기반 온보딩 진행률	로그인
POST	/onboarding/complete	온보딩 완료 표시	로그인

메서드	경로	설명	권한 스코프
GET	/dreams	출처와 상태를 포함한 개인 Dream 검토함 조회	dreams:read
POST	/dreams/{id}/feedback	노출·채택·숨김 등의 무중복 피드백 기록	dreams:read
POST	/dreams/{id}/accept	후보를 Dream 메모와 출처 연결선으로 확정	dreams:read, notes:write
POST	/dreams/{id}/regenerate	같은 출처에서 중복되지 않는 다른 관점 생성	dreams:read
POST	/dreams/{id}/develop	확장·반대 관점·실행 항목으로 발전	dreams:read
POST	/dreams/{id}/developed-note	발전 결과를 새 메모와 expanded 연결선으로 원자 저장(동일 재시도 무중복)	dreams:read, notes:write

GET /notifications 의 각 항목에는 resourceType , resourceId , resourceSpaceId , metadata 가 포함됩니다. Dream 알림은 /dreams?focus={resourceId} , 공간 공유 알림은 /space/{resourceId} , 댓글·멘션은 /space/{resourceSpaceId}?note={resourceId} 로 연결할 수 있습니다.

개인 설정과 API key 생성·회전·폐기, 모든 /admin/* 작업은 API key가 아니라 로그인한 브라우저 세션에서만 허용됩니다. 제한된 자동화 key가 새 자격 증명을 만들거나 관리자 role을 상속해 권한을 넓힐 수 없습니다.

/search , /notifications , /dreams , /admin/audit 는 응답의 nextCursor 를 다음 요청의 cursor 에 그대로 전달합니다. cursor 내부 형식에 의존하지 마세요.

하이브리드 검색은 접근 가능한 전체 메모에서 키워드 일치를 먼저 보존한 뒤 최근 비키워드 후보에 로컬 의미 점수를 결합합니다. 따라서 메모가 2,000개를 넘거나 검색어가 본문 2,000자 이후에 있어도 오래된 정확한 키워드 결과를 누락하지 않습니다.

🔗 서명 웹훅

/webhooks 에서 이벤트 subscription을 만들면 secret이 한 번만 반환됩니다. 대상은 공개 HTTPS 443이어야 하며 HMAC 입력은 아래와 같습니다.

```
signed = X-Umm-Timestamp + "." + raw_request_body
X-Umm-Signature-256 = "sha256=" + hex(HMAC-SHA256(secret, signed))
```

도메인 변경과 활성 구독별 PostgreSQL outbox는 같은 트랜잭션에서 확정되며, 프로세스가 재시작되어도 워커가 대기 항목을 이어서 처리합니다. 전달 시도는 at-least-once 방식이므로 수신 측은 X-Umm-Delivery 를 멍든 키로 사용해 중복을 무시하고 timestamp 허용 시간도 함께 확인해야 합니다. terminal payload는 즉시 비워지고 delivery metadata는 30일 보존됩니다. 이벤트에는 space.updated , note.* , edge.created , comment.* , member.* , dream.accepted 가 있습니다.

안전한 재시도와 오류

Canvas의 메모·연결·댓글 생성/수정/삭제 요청에 8~128자의 `Idempotency-Key` 를 보내면 같은 사용자와 키의 성공 응답을 24시간 재생합니다. 서버는 method·경로·query·본문 fingerprint가 같은 요청만 재생하며, 다른 요청에 키를 재사용하면 409입니다. 처리 중 2분 lease는 handler가 살아 있는 동안 갱신되고, 아직 처리 중이면 `Retry-After` 가 포함된 425를 반환합니다. 도메인 변경·SSE·webhook outbox·먹등 성공 응답은 같은 PostgreSQL 트랜잭션으로 확정되므로 성공 응답 기록 실패가 mutation 중복을 만들지 않습니다. handler 실행 전 프로세스가 중단되면 lease 만료 후 같은 요청이 예약을 자동 재획득합니다. 오프라인 클라이언트는 한 논리 작업에 같은 키를 유지하고 `Retry-After` 이후 다시 시도해야 합니다. 장시간 AI 요청과 API key·웹훅 서명 key처럼 비밀을 한 번만 공개하는 요청은 `Idempotency-Key` 를 거부합니다.

오류 본문은 `application/problem+json` 이며 `type`, `title`, `status`, `detail` 을 제공합니다. 기존 클라이언트를 위해 `error` 도 같은 설명을 유지합니다. 접근 가능한 메모의 version 충돌만 409와 `clientVersion`, 최신 서버 `latest` 메모를 반환합니다. 메모가 삭제되었거나 공간 접근 권한을 잃은 경우에는 404로 종료합니다.

브라우저 오프라인 queue는 409 충돌과 408·425·429·5xx처럼 다시 시도할 수 있는 응답만 보존하고 일시 오류는 `Retry-After` 또는 기본 5초 뒤 자동 재시도합니다. 400·404 등 영구적인 client 오류는 사용자에게 사유를 알린 뒤 queue에서 제거해 무한 재시도를 막습니다. 메모를 볼 수는 있지만 공간 권한이 `edit` 에서 `view` 로 낮아진 경우에는 `note-read-only` 403 Problem Details를 반환하며, 클라이언트는 일반 인증/권한 403과 구분해 적용 불가능한 변경만 제거합니다. flush 도중 다른 탭이나 새 편집이 추가한 항목은 최신 queue와 ID 단위로 병합하고 탭 간 Web Lock으로 저장을 직렬화해 진행 중이던 snapshot이 덮어쓰지 않습니다.

⚡ 실시간 이벤트 스트림 (SSE)

- 경로: `GET /api/v1/spaces/{spaceID}/events`
- 프로토콜: Server-Sent Events (SSE)
- 이벤트 타입: `space-change`
- 용도: 타 사용자가 캔버스에서 포스트잇을 추가/수정/이동할 때 실시간 브로드캐스트 수신

3. Model Context Protocol (MCP) 엔드포인트

- 엔드포인트: `POST /mcp`
- 인증: `Authorization: Bearer <API_KEY>`
- 프로토콜: JSON-RPC 2.0 (Stateless HTTP POST)

🔧 제공 도구 목록 (Tools)

1. `list_spaces`: 접근 가능한 공간 목록 조회
2. `list_notes`: 지정된 공간의 생각 포스트잇 및 연결선 조회
3. `create_note`: 캔버스에 새로운 생각 포스트잇 생성
4. `connect_notes`: 두 생각 간의 연결선 생성
5. `search_notes`: 지정 공간의 생각 검색
6. `get_related_notes`: 오프라인 유사도 기반 연관 생각 조회

7. `list_clusters`: 생각 군집 조회
8. `list_dreams`: 출처와 검토 상태를 포함한 최근 Dream 조회

💬 MCP 호출 예시 (JSON-RPC 2.0)

요청: 생각 포스트잇 생성

```
{
  "jsonrpc": "2.0",
  "id": "req-101",
  "method": "tools/call",
  "params": {
    "name": "create_note",
    "arguments": {
      "space_id": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
      "content": "MCP 엔드포인트를 통해 Cursor나 Claude에서 직접 생각을 붙이고 연결할 수 있습니다.",
      "color": "lavender",
      "x": 400,
      "y": 250
    }
  }
}
```

응답

```
{
  "jsonrpc": "2.0",
  "id": "req-101",
  "result": {
    "content": [
      {
        "type": "text",
        "text": "생각 포스트잇이 성공적으로 생성되었습니다. (ID: 9b1deb4d-3b7d-4bad-9bdd-2b0d7b3dcb6d)"
      }
    ]
  }
}
```